M T W T F S S
Page No.:
Date:
YOUVA

Assignment-2 Sparse Matrix Operations

Aim:- Write a C program for sparse matrix realization and operations on it - Transpose (Simple & Fast)

Objective:-

* To Study the concept of Sparse matrix, how it's stored and displayed
* To understand the implementation of sparse matrix operations - Simple Transpose, Fast Transpose

Implementation:-

Platform: open source C Programming tool like gcc.

Ps

Pseudo Code:-

Main Algorithm

1) Start
2) Input the matrix dimensions (rows, cols)
3) Call read() to populate the original matrix
4) Call convert() to generate the triplet compact matrix
5) Display compact matrix using display()
6) Display menu:
   * If choice = 1, call simple() & display()
   * If choice = 2, call fast() & display()
   * Else print "Invalid choice!"
7) Stop

Sub Algorithms

1) read (rows, cols, matrix)
   1. ① Iterate through rows from i=0 to rows-1
      ② Iterate through cols from j=0 to cols-1
      ③ Read ~~almost~~ element & store at matrix [i][j]

M T W T F S S
Page No.:
Date:
YOUVA

2) convert (rows, cols, matrix, compact)
① Set index k=1
② Traverse each element matrix [i][j] of the original matrix.
* If matrix [i][j] ≠ 0:
* Set compact[k][0] = i
* Set compact[k][1] = j
* Set compact[k][2] = matrix[i][j]
* Increment k by 1
③ Store header metadata of compact[0]:
* compact[0][0] = rows
* compact[0][1] = cols
* compact[0][2] = k-1

3) display (compact)
① Print table column headers
② Traverse index i from 0 to compact[0][2]
③ Print compact[i][0], compact[i][1] and compact[i][2] for each row.

4) simple (compact, trans)
① Read total cols = compact[0][1] & total Val = compact[0][2]
② Set header for trans
* trans[0][0] = total cols, trans[0][1] = compact[0][0]
* trans[0][2] = total Val
③ Initialize destination index, k=1
④ Iterate column loop from i=0 to total cols-1
* Iterate triplet terms from j=1 to total total Val
* If compact[j][1] == i:
* trans[k][0] = compact[j][1]
* trans[k][1] = compact[j][0]
* trans[k][2] = compact[j][2]
* Increment k by 1

5) fast (compact, trans)
1) Read columns = compact[0][1] and non_zeros = compact[0][2]
2) Set total[i] = 0 for all i from 0 to colums-1
3) Take each element i from 1 to non zeros, increment total[col_no], where col_no = compact[i][1]
4) * Set index[0] = 1
   * For i = 1 to columns set index[i] = index[i-1] + total[i-1]
5) trans[0][0] = colums, trans[0][1] = compact[0][0], trans[0][2] = trans[0][3] = non zeros
6) For each term i from 1 to non zeros
   * col_no = compact[i][1]
   * loc = index[col_no]
   * trans[loc][0] = compact[i][1]
   * trans[loc][1] = compact[i][0]
   * trans[loc][2] = compact[i][2]
   * Increment index[col_no]

Time Complexity:-

* Simple Transpose → O(cols x Non_Zeros) = O(n·t) where n = cols & t = Non-Zeros
* Fast Transpose → O(cols + Non_Zeros) O(n+t), where n = cols & t = Non-Zeros

Conclusion:- This implement sparse matrix Operations assignment. This system is able to perform different operations on sparse matrices such as simple and fast transpose and their time complexities

M T W T F S S
Page No.:
Date:
YOUVA

FAQs

1) What is a sparse Matrix? List the applications.

Ans A sparse matrix is a matrix in which the majority of elements are zero

Applications:
1. Network graph algorithms
2. Finite Element Analysis in engineering
3. ML and Natural language Processing
4. Image Processing

2) Represent spare matrices with suitable data structures. Explain with an example simple and fast transpose.

Ans A sparse matrix is commonly stored in compact or is Triplet form using a 2D array (size = (t+1) x 3, where t is no of non-zero elements)

* Row 0 (Header) → Total Rows Total Cols, Total Non-Zeros
* Row 1 to t → Row index, Column index, Value

eg: $\begin{bmatrix} 0 & 5 & 0 \\ 0 & 0 & 8 \\ 0 & 0 & 0 \end{bmatrix}_{(3\times3)}$

Triplet form →
3 3 2
0 1 5
1 2 8

Transpose Methods:

1. Simple Transpose: Scans column indices from 0 to (cols-1) sequentially. For every columns, it searches through all t non-zero entries to copy elements in row sorted order.

* Result:
3 3 2
1 0 5
2 1 8

Time complexity
O (Cols x Non-Zero)

2) Fast Transpose → * Calculate entry frequencies per column into array total[ ]

* Computes exact starting positions for each column using index[i] = index[i-1] + total[i-1]
* Places each element directly into its precalculated target location in a single pass.
* Result: Same as simple transpose, but in linear time

3) Find out the addition of two sparse matrices in triplet form and also find simple & Fast transpose?

Ans M1 =

4	5	6
0	3	5
1	3	8
1	4	43
2	3	4
3	2	45
4	1	2

M2 =

4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	4	21

Sum =

4	5	9
0	3	12
0	4	6
1	3	8
1	4	49
2	1	8
2	3	4
3	2	90
4	1	2
4	4	21

Transpose of M1:

5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	43

20/10/26

```
#include <stdio.h>

#define MAX 100
```

```
void read(int rows, int cols, int matrix[MAX][MAX]);
void convert(int rows, int cols, int matrix[MAX][MAX], int compact[MAX][3]);
void display(int compact[MAX][3]);
void simple(int compact[MAX][3], int trans[MAX][3]);
void fast(int compact[MAX][3], int trans[MAX][3]);

int main()
{
    int matrix[MAX][MAX];
    int compact[MAX][3];
    int trans[MAX][3];
    int rows, cols;
    int choice;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &rows, &cols);
    read(rows, cols, matrix);

    convert(rows, cols, matrix, compact);
    printf("\nCompact Form: ");
    display(compact);

    printf("\nChoose Transpose Method:\n");
    printf("1. Simple Transpose\n");
    printf("2. Fast Transpose\n");

    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);
```

```
    switch (choice)
    {
        case 1:  simple(compact, trans);
                 printf("\nResult: ");
                 display(trans);
                 break;

        case 2: fast(compact, trans);
                 printf("\nResult: ");
                 display(trans);
                 break;

        default: printf("\nInvalid choice!\n");
                  break;
    }

    return 0;
}

void read(int rows, int cols, int matrix[MAX][MAX])
{
    printf("Enter matrix elements: \n");
    for (int i=0; i<rows; i++)
    {
        for (int j=0; j<cols; j++)
          {
           scanf("%d", &matrix[i][j]);
        }
```

```
    }
}

void convert(int rows, int cols, int matrix[MAX][MAX], int compact[MAX][3])
{
    int k = 1;
    for (int i=0; i<rows; i++)
    {
        for (int j=0; j<cols; j++)
          {
           if (matrix[i][j]!=0)
             {
              compact[k][0] = i;
              compact[k][1] = j;
              compact[k][2] = matrix[i][j];
              k++;
             }
          }
    }
    compact[0][0] = rows;
    compact[0][1] = cols;
    compact[0][2] = k - 1;
}

void display(int compact[MAX][3])
{
    printf("\nRow\tCol\tValue\n");
    for (int i=0; i<=compact[0][2]; i++)
    {
```

```
        printf("%d\t%d\t%d\n", compact[i][0], compact[i][1], compact[i][2]);
    }
}

void simple(int compact[MAX][3], int trans[MAX][3])
{
    int totalCols = compact[0][1];
    int totalVal = compact[0][2];
    int k = 1;
    trans[0][0] = compact[0][1];
    trans[0][1] = compact[0][0];
    trans[0][2] = totalVal;
    for (int i=0; i<totalCols; i++)
    {
        for (int j=1; j<=totalVal; j++)
          {
           if (compact[j][1] == i)
             {
              trans[k][0] = compact[j][1];
              trans[k][1] = compact[j][0];
              trans[k][2] = compact[j][2];
              k++;
             }
        }
    }
}

void fast(int compact[MAX][3], int trans[MAX][3])
{
```

```
int loc, i, col_no;
int columns = compact[0][1];
int non_zeros = compact[0][2];
int total[MAX];
int index[MAX];
for (i=0; i<columns; i++)
{
    total[i] = 0;
}
for (i=1; i<=non_zeros; i++)
{
    col_no = compact[i][1];
    total[col_no]++;
}
index[0] = 1;
for (i=1; i<=columns; i++)
{
    index[i] = index[i - 1] + total[i - 1];
}
trans[0][0] = compact[0][1];
trans[0][1] = compact[0][0];
trans[0][2] = compact[0][2];
for (i=1; i<=non_zeros; i++)
{
    col_no = compact[i][1];
    loc = index[col_no];
    trans[loc][0] = compact[i][1];
    trans[loc][1] = compact[i][0];
    trans[loc][2] = compact[i][2];
```

```
        index[col_no]++;
    }
}
```

Output:

```
--- Matrix Operations Menu ---
1. Addition of two matrices
2. Subtraction of two matrices
3. Multiplication of two matrices
4. Transpose of a matrix
5. Exit
Enter your choice (1-5): 3
Enter rows and columns for Matrix A: 2 2
Enter rows and columns for Matrix B: 2 2

Enter elements of Matrix A:
Enter element [0][0]: 1
Enter element [0][1]: 2
Enter element [1][0]: 3
Enter element [1][1]: 4

Enter elements of Matrix B:
Enter element [0][0]: 5
Enter element [0][1]: 6
Enter element [1][0]: 7
Enter element [1][1]: 8

Resultant Matrix (A * B):
19      22
```

43 50

--- Matrix Operations Menu ---

1. Addition of two matrices

2. Subtraction of two matrices

3. Multiplication of two matrices

4. Transpose of a matrix

5. Exit

Enter your choice (1-5): 5

Exiting program!